

Assignment: Multi-Tenant Agentic Voice Orchestrator

Overview

Welcome to the AI Engineer assessment! In this assignment, you will build an end-to-end cloud-native system for a **Multi-Tenant Voice AI SaaS**.

Your system will allow multiple companies (tenants) to automatically call their customers for **Lead Qualification** using an AI voice agent. You will use **LangGraph** for agentic orchestration, **Vapi AI** for handling voice calls, **MongoDB** (or Cloud SQL/PostgreSQL) for persistence, a **lightweight frontend dashboard** for managers, and deploy the entire solution to **GCP Cloud Run** (preferred) or an alternative like **Render**.

 **Time Expectation:** You have **48 hours** to complete this assignment. We do not expect production-ready perfection, but rather a functional, well-architected prototype that demonstrates your understanding of agentic workflows, responsive full-stack implementation, and modern cloud deployment.

Business Scenario

Imagine a SaaS platform used by various real estate agencies (Companies). Each agency uploads a list of potential leads (Customers).

Your system needs to:

1. Provide an intuitive frontend to view agents, tenants, leads, and initiate outbound campaigns.
2. Identify pending leads for a specific company.
3. Programmatically initiate outbound voice calls through Vapi AI to these leads.
4. Converse with the lead to qualify them (e.g., "Are you looking to buy or sell a property?").
5. Handle webhooks securely on a public GCP Cloud Run endpoint.
6. Analyze the finished call transcript with LangGraph to determine the lead status.
7. Update the customer's status dynamically on the dashboard.

Core Requirements & Tasks

Task 1: Multi-Tenant Database Design

Design a schema using **MongoDB** (or PostgreSQL/Cloud SQL) to support a multi-tenant architecture.

- **Company (Tenant):** Needs an identifier, name, and specific prompt instructions (e.g., Company A sells houses, Company B rents apartments).

- **Customer (Lead):** Linked to a Company. Needs a name, phone number, and a status field (e.g., PENDING, CALL_INITIATED, QUALIFIED, NOT_INTERESTED, FAILED).
- **Call Logs:** To store transcripts, summaries, and call metadata.
- *Requirement:* Pre-seed your database with 2 mock companies and 2-3 mock customers each.

Task 2: Vapi AI Outbound Call Integration

Set up a free [Vapi.ai](https://vapi.ai) account and configure a Voice Assistant.

- Write an API handler to trigger an outbound call to a customer's phone number using the Vapi REST API.
- Pass dynamic context to Vapi (e.g., Customer Name, Company Name) so the AI sounds personalized.
- Configure the Vapi assistant to send webhooks back to your backend upon call completion.

Task 3: Agentic Orchestration with LangGraph

Use **LangGraph** to build the brain of the operation. Create a stateful graph with the following conceptual flow:

1. **Dispatch Node:** A node that fetches all PENDING customers for a given company_id, triggers the Vapi AI outbound call, and updates the database status to CALL_INITIATED.
2. **Evaluation Node:** A node triggered when a webhook is received from Vapi. It takes the call transcript/summary provided by Vapi, passes it to an LLM (OpenAI/Anthropic), and deduces the exact outcome.
3. **State Update Node:** Based on the Evaluation Node's reasoning, update the customer's status in the database (e.g., QUALIFIED or NOT_INTERESTED).

Task 4: Webhook Handler

- Build a lightweight API server (FastAPI, Flask, or Express/NestJS).
- Expose a secure webhook endpoint (e.g., /api/webhooks/vapi) to receive real-time updates from Vapi.
- When a call ends, this endpoint should route the payload into your LangGraph evaluation workflow to process the transcript and update the database.

Task 5: Lightweight Frontend Dashboard

Build a simple, responsive dashboard (React, Vue, or plain HTML/Tailwind) for company admins to manage their campaigns:

- **Tenant Selector:** Switch between different seeded mock companies.
- **Lead Directory:** View the list of leads for the selected company and their current calling statuses (PENDING, CALL_INITIATED, etc.).
- **Trigger Campaign Button:** A primary action button to programmatically initiate outbound calls to all PENDING leads for that specific company.

- **Live Update State:** Provide a way to manually refresh the status or use simple polling (or webhooks/websockets) to show changed lead statuses in real time.

Task 6: Cloud Deployment

Deploy your complete backend system (and frontend, if unified) to a cloud platform. **Google Cloud Run** is the preferred choice, but you may use alternatives like Render:

- **Containerization:** Create a multi-stage Dockerfile optimized for lightweight deployment.
- **Infrastructure Configuration:** Provide a CLI script or configuration file (e.g., `cloudbuild.yaml` or a shell script deploying using `gcloud run deploy`) for reproducible builds.
- **Secrets Management:** Use environment variables or GCP Secret Manager to securely pass sensitive credentials (such as Vapi keys, database URIs, and LLM API keys) to the Cloud Run environment.
- **Public Endpoint:** Configure the service to allow unauthenticated invocations so Vapi's webhook system can successfully post call logs back to your deployed backend.



Technical Constraints & Stack

- **Database:** MongoDB Atlas (M0 Free Tier) or GCP Cloud SQL (PostgreSQL).
- **AI Orchestration:** langgraph (Python or JS/TS). Python is strongly preferred.
- **Voice AI:** Vapi.ai API.
- **Backend API:** Python (FastAPI/Flask) or Node.js (Express/NestJS).
- **Frontend:** Tailwind CSS combined with React, Vue, Next.js, or vanilla JS.
- **Hosting:** GCP (Preferred) or other platforms like Render.
- **Testing:** Use standard environment variables (`.env`) for secrets. Do not commit keys to GitHub.



Bonus Points (If you have extra time)

- **Dynamic Prompting:** The LangGraph agent dynamically constructs the Vapi system prompt based on the specific Company's profile in the database before making the call.
- **Infrastructure-as-Code:** Automate your deployment via a script (e.g., shell script or Cloud Build YAML) to ensure reproducible builds on your chosen platform (GCP preferred).
- **Human-in-the-loop:** If the LLM is unsure about the call transcript, it sets the status to `NEEDS_REVIEW` instead of automatically categorizing it, alerting the admin on the frontend.



Deliverables

1. **Source Code:** A link to a public/private GitHub repository.
2. **Deployed URLs:** Links to the live, working frontend and backend running on GCP Cloud Run.
3. **Documentation (README.md):**
 - Clear instructions on how to set up environment variables (`.env`).

- Step-by-step instructions to build, run, and test the project locally.
 - A copy of your Dockerfile and a guide on deploying it to your own GCP project.
 - A brief explanation of your LangGraph architecture (nodes, edges, and state).
4. **Demo Video (Loom/YouTube):** A 3-5 minute video demonstrating:
- The frontend showing a specific tenant and pending leads.
 - Triggering the outbound campaign from the dashboard.
 - Your mobile phone receives the Vapi AI call (put it on speaker so we can hear the qualification conversation).
 - The Vapi webhook hitting your public Cloud Run server.
 - The dashboard dynamically updates to reflect the qualified status.



Evaluation Criteria

- **Cloud Deployment & Architecture (25%):** Correct, secure deployment. Emphasis on proper use of containerization and secret management (GCP deployment is preferred).
- **Agentic Logic & LangGraph (25%):** Effective use of LangGraph. Does the agent evaluate transcripts intelligently rather than using hardcoded logic?
- **Frontend Usability & Flow (20%):** A clean, simple dashboard layout. Correct multi-tenant separation on the UI and seamless campaign triggering.
- **Execution & Integration (20%):** Does the end-to-end calling workflow run reliably? Are webhook signatures or structures validated correctly?
- **Code Quality (10%):** Clean, organized repository with instructions. Readable, well-documented code.

Good luck! We are excited to see what you build.